hashlock.

Security Audit

Together Fun (dApp)

February 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

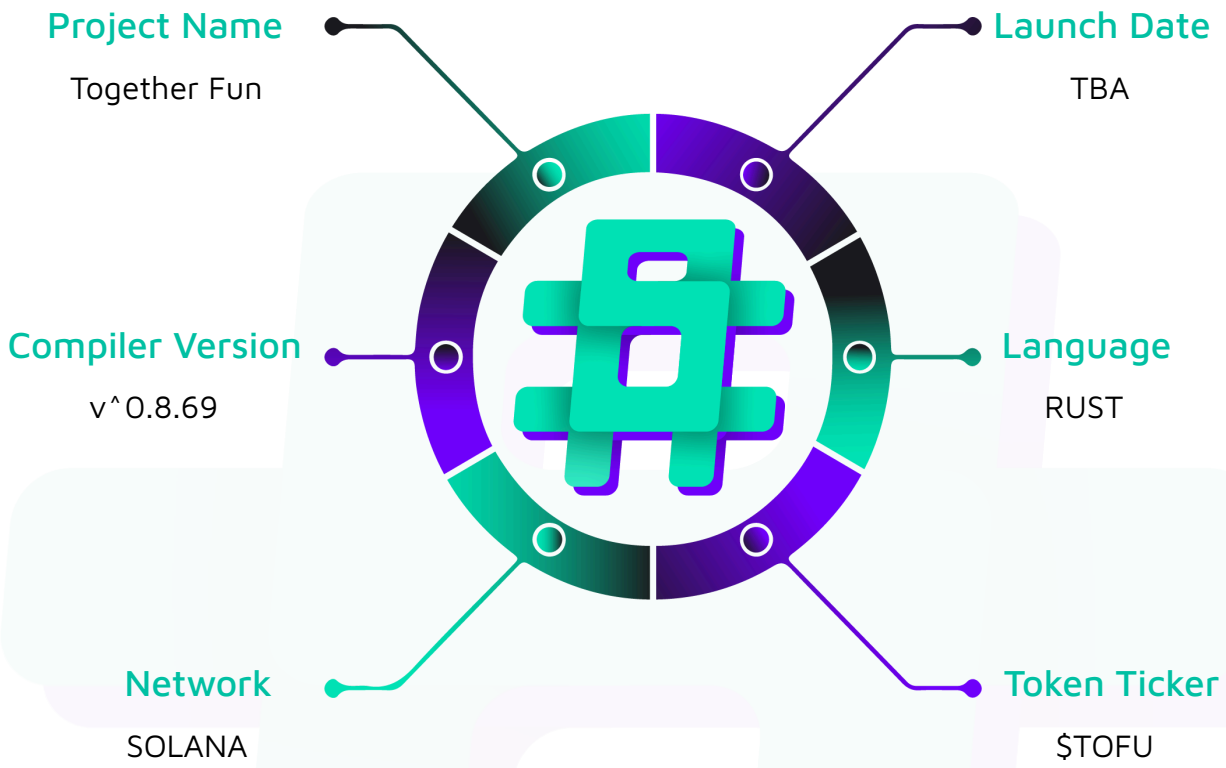
The Together Fun team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

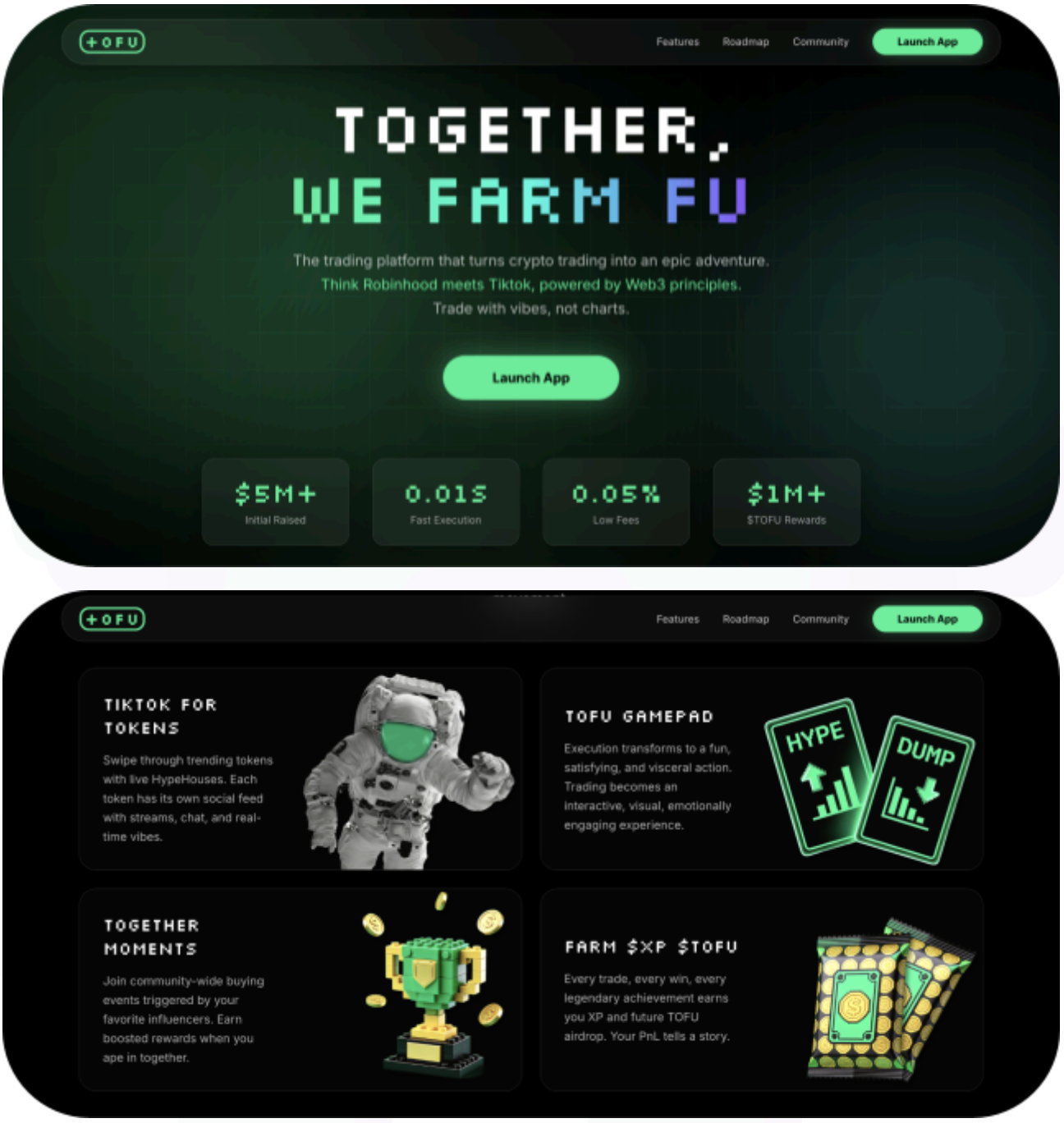
Together.fun is a gamified social trading platform designed to make cryptocurrency discovery and participation more engaging through community-driven interactions. The platform combines elements of social media and Web3, allowing users to discover tokens in a short-form, feed-based experience, participate in community events, and earn rewards such as $XP and $TOFU. By emphasizing real-time interaction, gamification, and incentive mechanisms, Together.fun aims to create a more interactive and community-centric approach to crypto trading and engagement.

Project Name: Together Fun
Project Type: dApp
Website: https://www.together.fun/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Together Fun project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Together Fun Smart Contracts**
Platform	**Ethereum / Rust**
Audit Date	**February, 2026**
Contract 1	consume_randomness.rs
Contract 2	equest_randomness.rs
Contract 3	claim_payout.rs
Contract 4	force_payout_batch.rs
Contract 5	deposit.rs
Contract 6	withdraw.rs
Contract 7	claim_refund.rs
Contract 8	force_refund_batch.rs
Audited MD5 hash	155ba2e52a6cbfdce67fa3417bb13185
Fix Review MD5 hash	cd880602a8fe32c15a60e1bb00d3c0da

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 High severity vulnerabilities

2 Medium severity vulnerabilities

5 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
consume_randomness.rs: - Receive 32-byte VRF callback - Stake-weighted sector selection - Transition room to Distributing	**Contract achieves this functionality.**
Request_randomness.rs: - Send VRF request to Oracle - Deduct the Oracle fee from the prepaid - Set the vrf_requested flag to true	**Contract achieves this functionality.**
Claim_payout.rs: - Compute the winner's pro-rata share - Transfer payout from vault - On the last claim, collect fees	**Contract achieves this functionality.**
force_payout_batch.rs: - Admin pays multiple winners - Create ATAs if needed - Finalize the room when complete	**Contract achieves this functionality.**
deposit.rs: - Accept SOL, wrap to wSOL - Create or top-up participant record - Update room sector stakes and counts	**Contract achieves this functionality.**

withdraw.rs: - Transfer wSOL back to user - Decrement room sector tracking - Close participant, return rent	**Contract achieves this functionality.**
claim_refund.rs: - Return full stake to the user - Decrement sector stakes and counts - Close participant account	**Contract achieves this functionality.**
Force_refund_batch.rs: - Admin refunds multiple participants - Create ATAs if needed - Close accounts, drain lamports	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Together Fun project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Together Fun project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] request_randomness - Missing lock phase after request_randomness

Vulnerability Details

The `request_randomness` sets `room.vrf_requested = true` but does not change `room.status` away from `Gathering`. The three user-action instructions that mutate the game state only gate on status:

- `deposit` requires `room.status == Gathering` - no `!vrf_requested` check.
- `withdraw` requires `room.status == Gathering` - no `!vrf_requested` check.
- `change_sector` requires `room.status == Gathering` - no `!vrf_requested` check.

What is important, `consume_randomness` receives `randomness: [u8; 32]` as a visible instruction argument. This value is observable in the Solana mempool before the transaction is confirmed. An attacker, especially a validator or MEV bot, can:

1. Observe the pending `consume_randomness` transaction.
2. Compute which sector will win using the same weighted-selection algorithm.
3. Submit a front-running `change_sector`, or `withdraw` + `deposit` transaction with higher priority.
4. Move their entire stake to the winning sector before the randomness is consumed.

Even without mempool front-running, the window between `request_randomness` and `consume_randomness` allows legitimate users to change their positions after the game was supposed to be "locked", breaking fairness assumptions.

Impact

A motivated participant can front-run the VRF callback to manipulate the game outcome moving stakes to the winning sector, withdrawing from the losing sector, or altering stake weights to shift the probability distribution after the randomness value is already deterministic. This completely breaks the fairness guarantee of the game.

Recommendation

Introduce a locked state. Add a `RoomStatus::VrfPending` status set in `request_randomness`, and require `status == Gathering` (not `VrfPending`) in `deposit`, `withdraw`, and `change_sector`.

Status

Resolved

[H-02] claim_payout - Missing round validation in claim_payout

Vulnerability Details

The `claim_payout` validates that the room is in `Distributing` status, the participant's wallet matches the signer, and `!participant.payout_claimed`, but it never checks `participant.round == room.current_round`.

The participant PDA is derived from `[PARTICIPANT_SEED, room.key(), user.key()]`, and it does not include the round number. Combined with `restart_room` not requiring all previous participants to be closed (another issue in the report), stale participant accounts survive across rounds.

Potential attack path:

1. Round 1 - Alice deposits 1 SOL to sector 0. Round 1 ends - Alice is a loser whose participant account is never closed.
2. Room restarts to Round 2. Alice's participant persists with `sector = 0`, `stake_amount = 1 SOL`, `round = 1`.
3. Round 2 - Bob deposits 100 SOL to sector 0, Carol deposits 100 SOL to sector 1. Sector 0 wins.
4. Alice calls `claim_payout`. Her stale participant has sector 0 (the winner). The payout is computed using her `stake_amount` of 1 SOL against `winner_sector_total` which reflects only Round 2's 100 SOL, not Alice's 1 SOL since `sector_stakes` was reset. Alice extracts a share she's not entitled to.

For `Default8Of9` mode, where 8 out of 9 sectors are winners, a stale participant has an ~89% chance of being on a winning sector in any given round.

Impact

A participant account from a previous round can claim a pro-rata share of the current round's pot without having deposited in that round. This directly steals funds from legitimate current-round winners.

Recommendation

We recommend adding an assertion to the `ClaimPayout` account constraints or at the start of the handler:

```
require!(participant.round == room.current_round, ErrorCode::InvalidRound);
```

Status

Resolved

[H-03] claim_refund - Stale participants can drain the refund vault

Vulnerability Details

The `claim_refund` checks that `room.status == Cancelled` and `participant.wallet == user.key()`, but never verifies `participant.round == room.current_round`.

Potential attack path:

1. Round 1 - Bob deposits 5 SOL. Round 1 is complete, but Bob's loser account is never closed.
2. Room restarts to Round 2. Charlie deposits 10 SOL.
3. Round 2 is cancelled. Vault holds 10 SOL (Charlie's deposit).
4. Bob calls `claim_refund`. His stale participant has `stake_amount = 5 SOL`. The vault transfers 5 SOL to Bob.
5. Charlie calls `claim_refund` for his 10 SOL, but only 5 SOL remain. Charlie loses half his funds.

Impact

If a later round is cancelled, a stale participant from a prior round can claim their old `stake_amount` as a refund from the new round's vault, stealing funds from current-round participants.

Recommendation

We recommend adding an assertion to the `ClaimPayout` account constraints or at the start of the handler:

```
require!(participant.round == room.current_round, ErrorCode::InvalidRound);
```

Status

Resolved

[H-04] consume_randomness - Cancelled room can be resurrected into distributing

Vulnerability Details

The `consume_randomness` only checks:

- `room.vrf_requested == true`
- `room.received_randomness.is_none()`

It does not verify that `room.status == Gathering`. Meanwhile, `cancel_game` sets `room.status = Cancelled` but does not clear `room.vrf_requested`.

If the sequence is `request_randomness` > `cancel_game` > VRF oracle delivers callback, then `consume_randomness`succeeds, sets `room.status = Distributing`, and selects a `winner_sector`. Participants who expected refunds (the room was cancelled) now find:

- `claim_refund` is blocked as it requires `status == Cancelled`.
- Losers lose their stakes unexpectedly.
- The admin's cancellation is silently overridden.

Impact

A room that was intentionally cancelled can have its status overridden to `Distributing` by a late-arriving VRF callback, permanently trapping refundable funds in a distribution flow nobody intended.

Recommendation

We recommend adding such an assert:

```
require!(room.status == RoomStatus::Gathering, ErrorCode::InvalidRoomState);
```

Status

Resolved

[H-05] deposit - Zero-amount deposit creates permanently stuck participants that block game finalization

Vulnerability Details

The `deposit` does not validate `amount > 0`. A zero-amount deposit:

- Creates a participant with `stake_amount = 0`.
- Increments `room.sector_participant_counts[sector]`.

If this participant ends up in the winning sector:

- Cannot `claim_payout` as the handler requires `payout_amount > 0`, which will be 0 for a zero-stake participant.
- Cannot be batch-processed as `force_payout_batch` skips participants with `payout_amount == 0`.
- Cannot be closed as a loser as `close_loser_participant` requires `!is_winner`, but this participant IS in the winning sector.
- Cannot withdraw as `withdraw` requires `withdrawal_amount > 0`.

The participant is permanently stuck. `participants_claimed` never reaches `expected_claims`, so the room never transitions to `Finished`, fees are never collected, and the vault is locked forever.

Additionally, zero-amount deposits can fill sector capacity (`MAX_PARTICIPANTS_PER_SECTOR = 100`) without risking any capital, griefing legitimate users from joining.

Impact

A single zero-amount deposit to the winning sector creates a participant that cannot be claimed, batch-processed, or closed permanently, preventing the room from reaching `Finished` status and locking the vault.

Recommendation

We recommend adding an assert in the deposit logic:

```
require!(amount > 0, ErrorCode::InvalidAmount);
```

Status

Resolved

Medium

[M-01] restart_room - The restart_room does not verify all participant accounts are closed

Vulnerability Details

The `restart_room` checks only that `vault.amount == 0` and the room is in `Finished` or `Cancelled`. It resets `sector_participant_counts` to all zeros, `participants_closed` to 0, etc., but does not verify that all participant accounts from the previous round have been closed.

The participant PDA is `[PARTICIPANT_SEED, room.key(), user.key()]` and it doesn't include the round number. Any unclosed participant accounts from the previous round survive into the new round with stale `round`, `sector`, and `stake_amount` values.

```
pub fn restart_room(ctx: Context<RestartRoom>) -> Result<()> {

    let authority_key = ctx.accounts.authority.key();

    let is_creator = ctx.accounts.room.creator == authority_key;

    let is_admin = ctx.accounts.config.can_manage_rooms(&authority_key);


    require!(is_creator || is_admin, ErrorCode::Unauthorized);


    require!(

        ctx.accounts.vault.amount == 0,

        ErrorCode::VaultNotEmpty

    );
```

Impact

Stale participants could persist across rounds.

Recommendation

Before allowing a restart, require that all participants from the previous round were closed:

```
let total_participants: u32 = room.sector_participant_counts.iter().sum();

require!(room.participants_closed >= total_participants, ErrorCode::ParticipantsNotClosed);
```

Status

Resolved

[M-02] force_payout_batch - The force_payout_batch confiscates small payouts to the calling authority

Vulnerability Details

In `force_payout_batch`, when `!ata_exists && payout_amount <= ata_rent`:

1. The full `payout_amount` is transferred from the vault to `authority_wsol_ata` (the caller's token account).
2. The participant account is zeroed and drained.
3. A `PayoutClaimed` event is emitted with `payout_amount: 0` for the user.

The calling authority, such as a room creator or admin, directly profits from users with small balances who lack a pre-existing wSOL ATA. For micro-stake games with many small participants, this can accumulate into material theft. The ATA rent threshold (~0.00203 SOL / 2,039,280 lamports) is not negligibly small.

It is worth mentioning that the same issue also occurs in the refund path.

Impact

When a winner has no wSOL ATA, and their payout is ≤ ATA rent (~0.00203 SOL), the calling authority receives the user's entire payout. The participant is closed, so the user has no recourse.

Recommendation

Do not redirect to authority. Instead, transfer the small payout as native SOL to the user's system account, bypassing ATA entirely.

Status

Acknowledged

Low

[L-01] claim_payout/force_payout_batch - Platform fee read from mutable global config during distribution

Description

Both `claim_payout` and `force_payout_batch` compute `platform_fee` from the current value of `config.fee_config.payout_fee_bps` at execution time. There is no per-room or per-round snapshot.

Potential attack scenario:

1. Fee is 200 bps (2%). Winner A claims. Their `distributable_pot` is computed with a 2% fee.
2. Admin raises fee to 800 bps (8%).
3. Winner B claims. Their `distributable_pot` is smaller (8% fee). But the vault has already given more to A.
4. When all claims are complete, the code tries to transfer the 8% fee from the vault, but the vault is short because A was paid using the 2% rate.

The same issue affects `creator_fee_bps` only partially, as it's stored per-room, but `payout_fee_bps` is global.

Recommendation

Snapshot fee parameters into the `Room` account at creation time or at `request_randomness` time.

Status

Acknowledged

[L-02] create_room - The creator_fee_bps is validated against a hardcoded max instead of a configurable max_creator_fee_bps

Description

In `create_room` there is a check validating if `creator_fee_bps > MAX_FEE_BPS`. This checks against the compile-time constant `MAX_FEE_BPS = 1000`. The runtime-configurable `config.fee_config.max_creator_fee_bps` is never referenced anywhere in `create_room`.

If an admin sets `max_creator_fee_bps` to 200 (2%), creators can still set 1000 (10%).

```
pub fn create_room(
    ctx: Context<CreateRoom>,
    room_type: RoomType,
    room_lifetime: RoomLifetime,
    creator_fee_bps: u16,
) -> Result<()> {
    if creator_fee_bps > MAX_FEE_BPS {
        msg!(
            "CreatorFeeExceedsMax: fee {} bps exceeds max {} bps (10%)",
            creator_fee_bps,
            MAX_FEE_BPS
        );
        return Err(ErrorCode::CreatorFeeExceedsMax.into());
    }
```

Recommendation

We recommend implementing an assert:

```
require!( creator_fee_bps <= config.fee_config.max_creator_fee_bps, ErrorCode::CreatorFeeExceedsMax );
```

Status

Resolved

[L-03] request_randomness - The request_randomness missing sector occupancy pre-check wastes the oracle fee and stalls the room

Description

The `consume_randomness` enforces `sector_stakes.len() >= 2`, where at least two sectors must have non-zero stakes. However, `request_randomness` has no equivalent pre-check. If randomness is requested with only one occupied sector:

1. VRF oracle delivers a callback.
2. `consume_randomness` rejects it (`InsufficientOccupiedSectors`).
3. Error reverts all state changes, so `received_randomness` stays `None`.
4. But `vrf_requested` was set to `true` in `request_randomness`
5. `request_randomness` checks `require!(!room.vrf_requested)` and block any retry.
6. The room is permanently stuck. The oracle fee deducted from `oracle_prepaid` is lost.

Recovery requires cancellation, losing the room state, and requiring a fresh round.

Recommendation

Add the sector check to `request_randomness`:

```
let occupied = room.sector_stakes.iter().filter(|&&s| s > 0).count();

require!(occupied >= 2, ErrorCode::InsufficientOccupiedSectors);
```

Status

Resolved

[L-04] deposit - Anyone can block a specific user from depositing

Description

The participant PDA is deterministically derived from `[PARTICIPANT_SEED, room.key(), owner.key()]`. On Solana, anyone can transfer SOL to any address, including PDAs. Transferring even 1 lamport to a PDA creates a system-owned account with lamports but zero data allocation.

When the victim tries to `deposit`, Anchor's `init_if_needed` calls `system_instruction::create_account` via `invoke_signed`. The system program's `create_account` fails if the target account already has lamports and the account isn't the funding source. The transaction reverts, and the victim cannot deposit.

The attack cost is 1 lamport + transaction fee (~5000 lamports), making it extremely cheap to grief specific users.

Recommendation

Implement PDA reclaim logic - if the account exists but is system-owned with zero data, use `invoke_signed` to allocate space and assign ownership directly.

Status

Acknowledged

[L-05] force_refund_batch - The force_refund_batch never transitions room to finished

Description

The `force_payout_batch` includes completion logic - when `participants_claimed >= expected_claims`, it sets `room.status = Finished`, enabling subsequent `finalize_game`. `force_refund_batch` has no equivalent completion logic. After processing all refunds, the room stays `Cancelled`.

The room can potentially be restarted as `restart_room` accepts `Cancelled`, but only if `vault.amount == 0`. If rounding dust remains, even a restart is blocked, and the vault rent (~0.002 SOL) is permanently locked.

```
pub fn restart_room(ctx: Context<RestartRoom>) -> Result<()> {

    let authority_key = ctx.accounts.authority.key();

    let is_creator = ctx.accounts.room.creator == authority_key;

    let is_admin = ctx.accounts.config.can_manage_rooms(&authority_key);


    require!(is_creator || is_admin, ErrorCode::Unauthorized);


    require!(

        ctx.accounts.vault.amount == 0,

        ErrorCode::VaultNotEmpty

    );
```

Recommendation

Add completion logic to `force_refund_batch`:

```
let total_participants: u32 = room.sector_participant_counts.iter().sum();

if room.participants_claimed >= total_participants {

    room.status = RoomStatus::Finished;
```

```
}
```

Status

Resolved

Centralisation

The Together Fun project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Together Fun project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

s

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au